
Production Foundations Learning Journey

Linux, curl, command-line tools, VMs, SSH, cron, Docker, Nginx, and the habits that connect them



A hands-on course for a beginner using one disposable Ubuntu VM on
Windows

July 2026

How to use this book

This is a journey, not a dictionary. Each chapter gives you a small mental model, one safe command at a time, and a checkpoint that proves what you understood. Keep an Ubuntu virtual machine open while you read. Make a snapshot before any lesson that changes networking, SSH, firewall rules, or services. The examples use Linux syntax. Your output will vary with your username, IP address, package versions, and current time; learn to recognize the shape of the result instead of memorizing one exact line.

Every practical example follows the same rhythm: problem, command, explanation, expected output, interpretation, common mistake, and a small exercise. If a command feels mysterious, stop after the explanation and run it again with one option removed.

The route through the course

1. Create a safe Linux lab.
2. Learn the terminal and use curl to make your first request.
3. Create, find, edit, transform, and move files.
4. Inspect the machine and understand permissions.
5. Diagnose a network from the URL outward.
6. Connect to another machine with SSH.
7. Schedule and supervise recurring work.
8. Package an application with Docker.
9. Put Nginx in front of it and deploy it carefully.
10. Finish by operating a small service and explaining every layer.

Contents

How to use this book	i
The route through the course	ii
1 Your first safe Linux lab	1
1.1 What the terminal actually is	1
1.2 Create the Ubuntu VM	1
1.3 First checkpoint	2
2 curl: see a real request	3
2.1 The smallest useful request	3
2.2 Read the flags one at a time	3
2.3 Save, inspect, and test	4
2.4 JSON APIs with jq	4
3 The everyday command-line toolbox	5
3.1 Files, paths, and safe movement	5
3.2 Search and transform text	5
3.3 Nano and Vim	5
3.4 FFmpeg and ffprobe	6
3.5 A small pipeline	6
4 Users, permissions, processes, and logs	7
4.1 Read permission bits	7
4.2 sudo and packages	7
4.3 Processes and signals	7
4.4 Logs are evidence	8
5 Networking from the URL outward	9
5.1 Name, address, route, socket	9
5.2 A diagnostic sequence	9
5.3 VM networking and UFW	9
6 SSH: operate another machine	11
6.1 Keys, locks, and fingerprints	11
6.2 Connect and verify	11
6.3 Copy files and use an agent	11
7 Services, cron, and reliable schedules	13
7.1 A systemd service	13
7.2 Cron basics	13
7.3 systemd timers and overlap protection	14

8 Docker: package the application	15
8.1 Image, container, volume, network	15
8.2 A Dockerfile, line by line	15
8.3 Compose and persistence	16
9 Nginx, HTTPS, and the edge	17
9.1 Reverse proxy mental model	17
9.2 A minimal server block	17
9.3 TLS and secrets	18
10 Git, deployment, and observability	19
10.1 A small Git loop	19
10.2 Health, readiness, and logs	19
10.3 Backups and rollback	19
11 Capstone: a small service from laptop to production shape	20
11.1 Build the application	20
11.2 Containerise and compose	20
11.3 Put Nginx at the edge	20
11.4 Schedule an operational task	20
11.5 Fault-injection checklist	20
12 Worked examples: command to evidence	22
12.1 Example: a health check that fails correctly	22
12.2 Example: turn an API response into a small report	22
12.3 Example: diagnose a port in layers	23
12.4 Example: make a scheduled job observable	23
12.5 Example: inspect a failed container	23
12.6 Example: validate an Nginx change	24
13 Practice labs	25
13.1 Lab 1: build a command diary	25
13.2 Lab 2: compare editors	25
13.3 Lab 3: a media inspection report	25
13.4 Lab 4: find a slow process	25
13.5 Lab 5: SSH between two terminals	25
13.6 Lab 6: cron versus systemd timer	26
13.7 Lab 7: container boundary	26
13.8 Lab 8: reverse proxy evidence	26
13.9 Lab 9: restore rehearsal	26
13.10 Lab 10: clean-snapshot capstone	26
A Command cards	27
B Troubleshooting map	28
C A thirty-day practice path	29

D Glossary and official references	30
Closing note	31

Your first safe Linux lab

Before learning commands, you need a place where mistakes are cheap. A virtual machine is a computer made of files, running inside your Windows computer.

1.1 What the terminal actually is

The graphical desktop is one way to ask the operating system for work. The terminal is another. You type a command; a shell reads it, finds a program, supplies arguments, and displays output. Bash is one shell; Zsh and PowerShell are other shells with different syntax. When you see a prompt such as `alice@ubuntu:~\`, the pieces are clues: `alice` is the user, `ubuntu` is the machine, the tilde means the home directory, and the dollar sign says this is an ordinary user prompt. A prompt ending in a hash sign usually means a privileged root shell.

```
whoami
hostname
printf 'The shell is running a command.\n'
```

Read the command: three tiny commands

`whoami` asks which user owns this shell. `hostname` prints the machine name, useful when several terminals look similar. `printf` writes exactly the text you provide; the backslash-n escape requests a new line. None of these commands changes files.

1.2 Create the Ubuntu VM

Install VirtualBox, download an Ubuntu LTS desktop ISO, and create a VM with roughly two virtual CPUs, 4–8 GB of RAM, and a dynamically allocated disk of at least 25 GB. Exact values depend on your laptop. Start with NAT networking. NAT lets the VM reach the internet while keeping inbound connections from the internet out of the VM by default. During installation, create a normal user with a strong passphrase. Use `sudo` only when a task genuinely needs administrator rights. That separation is part of the lesson: a command that can rewrite the whole machine should not be the default identity.

Take a VM snapshot named *fresh-install* now. Take another named *after-ssh* before changing SSH, firewall, or network settings. A snapshot is not a backup, but it is a fast way to return to a known practice state.

1.3 First checkpoint

```
mkdir -p ~/journey/{notes,downloads,lab}  
ls -la ~/journey
```

The `-p` option allows `mkdir` to create missing parent directories. Braces are expanded by Bash into three names before `mkdir` runs. `ls -la` means “list, including hidden entries, in a detailed format.”

You can identify the user, identify the VM, create a course workspace, and explain why the VM is a safer place to practise privileged commands.

curl: see a real request

curl is the first tool in the journey because it makes the invisible visible. You can ask a server for a page, inspect headers, save a response, and turn the result into a test.

2.1 The smallest useful request

```
curl https://example.com
```

Read the command: the URL and default output

curl is the program. The URL has a scheme (`https`), a host (`example.com`), and an optional path. With no output option, curl writes the response body to the terminal. It transfers bytes; it does not open a graphical browser.

Try a response that is easier to read:

```
curl -i https://example.com
```

The short option `-i` means “include response headers.” The first line is the HTTP status, followed by headers such as content type, then a blank line, then the body.

2.2 Read the flags one at a time

```
curl -I https://example.com
curl -v https://example.com
curl -sS https://example.com
curl -fsS https://example.com/health
```

- `-I` sends a HEAD request and asks for headers without the normal body. Some servers handle HEAD differently.
- `-v` enables verbose diagnostics: DNS, connection, TLS negotiation, request headers, and response headers.
- `-sS` hides progress output but still shows errors, keeping scripts readable.
- `-f` returns a failure exit code for HTTP 400 and later. The transfer may have worked while the application request failed.

`-k` disables TLS certificate verification. Use it only for a deliberately local test with a self-signed certificate; fix trust failures in production.

2.3 Save, inspect, and test

```
curl --fail --location --output ~/journey/downloads/page.html \  
  https://example.com  
wc -c ~/journey/downloads/page.html  
head -n 5 ~/journey/downloads/page.html
```

Read the command: download options

`-fail` makes HTTP errors visible to automation. `-location` follows redirects. `-output path` writes the body to a named file. The trailing backslash joins the next line into one command.

```
curl --fail-with-body --silent --show-error \  
  --max-time 10 --retry 3 https://example.com/health  
echo "curl exit code: $?"
```

The question mark expression reads the previous command's exit status. Zero means curl completed successfully. A non-zero value tells a script to take a failure path. The timeout and retries do not make a broken application healthy.

2.4 JSON APIs with jq

```
curl --fail --silent --show-error \  
  https://api.github.com/repos/curl/curl \  
  | jq '{name, stars: .stargazers_count, language}'
```

The pipe sends curl's output to jq. The jq expression constructs a smaller object from named JSON fields. This is safer than grepping JSON because jq understands data types and nesting.

Request <https://httpbin.org/status/404> with and without `-f`. Observe the body, the terminal error, and `\$?`. Then request <https://httpbin.org/redirect/1> with and without `-location`. Explain why the results differ.

You can separate a URL, request, response headers, body, and exit status. You can write a curl health check that fails loudly without leaking progress noise into logs.

The everyday command-line toolbox

The shell becomes powerful when small programs cooperate. Learn each tool as a narrow instrument, then combine them with pipes and redirection.

3.1 Files, paths, and safe movement

```
cd ~/journey/lab
printf 'first line\nsecond line\n' > notes.txt
cp notes.txt notes-copy.txt
mv notes-copy.txt archive.txt
ls -l
```

The greater-than sign redirects output into a file and replaces an existing file. Two greater-than signs append instead. `cp` creates a second file; `mv` changes its name or location. The `-l` form of `ls` shows mode bits, owner, size, and modification time.

Before using `>` or `rm`, print the path with `pwd` and `ls`. Practise destructive commands only in a disposable folder.

3.2 Search and transform text

```
rg --line-number 'second' ~/journey/lab
find ~/journey -type f -name '*.txt'
cat notes.txt | tr 'a-z' 'A-Z'
sort notes.txt | uniq -c
```

`rg` searches file contents and shows matching line numbers. `find` searches the filesystem using predicates. `tr` translates characters. `sort` orders lines so `uniq -c` can count adjacent duplicates.

3.3 Nano and Vim

Nano is a modeless editor. Open a file with `nano notes.txt`; type normally; press `Ctrl+O` to write and `Ctrl+X` to exit. The caret symbol in Nano's help means the Control key. Vim is modal. Start with `vim notes.txt`. Press `i` to insert, Escape for Normal mode, type `:w` and Enter to save, and

:q and Enter to quit. :wq saves and quits; :q! quits without saving.

Vim separates navigation from insertion. Learn insert, escape, save, quit, search, and undo first; postpone advanced commands.

3.4 FFmpeg and ffprobe

```
ffprobe -hide_banner recording.mp4
ffmpeg -i recording.mp4 -vn -c:a copy audio.m4a
ffmpeg -i recording.mp4 -vf scale=1280:-2 -c:v libx264 -crf 23 preview.mp4
```

ffprobe reports streams, codecs, dimensions, duration, and metadata. -i names the input, -vn removes video, and -c:a copy copies audio without re-encoding. The scale filter preserves aspect ratio; CRF controls H.264 quality. Re-encoding takes time and can reduce quality.

3.5 A small pipeline

```
curl --fail --silent https://example.com/robots.txt \
  | tee ~/journey/notes/robots.txt \
  | wc -l
```

tee writes the stream to a file and also passes it onward. The final wc -l counts lines. Observe each boundary before deciding whether the next tool received what you intended.

You can edit a file in Nano, save and quit in Vim, search with rg, inspect media with ffprobe, and explain where a pipe sends data.

Users, permissions, processes, and logs

A production machine is shared by people and programs. Linux permissions, process ownership, and logs are the language that prevents one task from silently damaging another.

4.1 Read permission bits

```
ls -l ~/journey/lab/notes.txt
chmod u+x ~/journey/lab/notes.txt
stat ~/journey/lab/notes.txt
```

The first character of the long listing describes the type. The next three groups describe permissions for the owner, group, and everyone else. `r` reads, `w` writes, and `x` executes or enters a directory. `chmod u+x` adds execute permission for the user without disturbing other bits.

4.2 sudo and packages

```
sudo apt update
apt list --upgradable
sudo apt install jq tree
command -v jq
```

`sudo` asks for temporary administrative authority. `apt update` refreshes package metadata; it does not upgrade packages. `apt install` resolves and installs the named packages. `command -v` shows which executable the shell will run.

4.3 Processes and signals

```
sleep 120 &
jobs -l
ps -ef | rg '[s]leep 120'
kill PID
```

The ampersand starts a background command. `jobs -l` shows shell-managed jobs and their process IDs. Replace `PID` with the actual number. `kill` sends `SIGTERM` by default, asking a

cooperative process to stop; SIGKILL is a last resort.

4.4 Logs are evidence

```
journalctl -b --no-pager | tail -n 30  
df -h  
free -h
```

`journalctl -b` limits the system journal to the current boot. `-no-pager` prints directly. `df -h` reports filesystem capacity; `free -h` reports memory. Inspect the machine before changing application code.

You can explain who owns a file, what a process is doing, and which command shows logs, disk, or memory evidence.

Networking from the URL outward

When a request fails, walk from the name to the wire: URL parsing, DNS, route, socket, TLS, HTTP, and application data. Each layer has different evidence.

5.1 Name, address, route, socket

```
ip addr
ip route
getent hosts example.com
ss -tulpn
```

`ip addr` lists interfaces and addresses. `ip route` shows where packets go. `getent hosts` asks the system resolver for a name. `ss -tulpn` lists listening TCP and UDP sockets.

5.2 A diagnostic sequence

```
ping -c 3 1.1.1.1
dig example.com
nc -vz example.com 443
curl -vI https://example.com
```

Ping tests one kind of reachability and may be blocked even when HTTPS works. `dig` shows DNS answers and timing. `nc -vz` attempts a TCP connection without transferring application data. `curl` then shows the TLS and HTTP layers.

5.3 VM networking and UFW

NAT is convenient for outbound access. Bridged mode gives the VM an address on the same LAN as Windows when another device must reach a service in the VM. Before switching modes, record the current address and take a snapshot.

```
sudo ufw status verbose
sudo ufw allow OpenSSH
sudo ufw enable
```

Never enable a firewall on a remote machine before allowing your management path. Test SSH in a second terminal before closing the first session.

You can tell whether a failure is probably DNS, a route, a closed port, TLS, HTTP, or an application response.

SSH: operate another machine

SSH turns a network connection into a remote terminal. It is the bridge from your practice VM to a cloud server, another VM, a Git host, or a production machine.

6.1 Keys, locks, and fingerprints

```
ssh-keygen -t ed25519 -C "journey@example.com"  
cat ~/.ssh/id_ed25519.pub  
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/id_ed25519
```

The public key is safe to place on a server; the private key is secret. Ed25519 is a modern compact key type. The passphrase protects the private key if the file is copied. The directory and file modes prevent other local users from reading key material.

6.2 Connect and verify

```
ssh ubuntu@192.168.56.20  
ssh -o ConnectTimeout=5 ubuntu@192.168.56.20 'hostname; uptime'
```

The first command opens an interactive shell. The second runs a remote command and returns to the local shell. On first connection, SSH shows the server fingerprint. Verify it through a trusted channel before accepting it.

6.3 Copy files and use an agent

```
scp notes.txt ubuntu@192.168.56.20:~/journey/  
rsync -av --dry-run ~/journey/ ubuntu@192.168.56.20:~/journey/  
ssh-add ~/.ssh/id_ed25519
```

scp performs a simple copy over SSH. `rsync -av --dry-run` previews a recursive archive-mode synchronization without changing the destination. Remove `--dry-run` only after the file list is correct. `ssh-add` loads a private key into the agent.

Use keys with passphrases, verify fingerprints, restrict permissions, use a host alias in `~/.ssh/config`, and keep a second recovery session while changing SSH settings.

You can distinguish a public key from a private key, make a safe first connection, and preview a file transfer.

Services, cron, and reliable schedules

A command you ran once is not yet a service. Reliability comes from a clear owner, a schedule, logs, timeouts, locking, and an alert when the expected result is missing.

7.1 A systemd service

```
systemctl status ssh --no-pager
systemctl list-units --type=service --state=running
journalctl -u ssh -b --no-pager
```

`systemctl status` shows state, recent logs, the unit file, and the main process. `list-units` provides an inventory. `journalctl -u ssh` filters evidence to one service.

7.2 Cron basics

Cron fields are minute, hour, day of month, month, and day of week. A schedule is only one part of a job; use absolute paths, explicit environment variables, a log destination, a timeout, and overlap protection.

```
crontab -l
crontab -e
```

For a safe practice job, add:

```
*/5 * * * * /usr/bin/date >> /home/ubuntu/journey/notes/cron.log 2>&1
```

The first field means every five minutes. The other four asterisks mean every hour, day, month, and weekday. The absolute path avoids relying on cron's smaller `PATH`. `2>&1` combines standard error with standard output.

This phrase can mean a local crontab editor, a host dashboard that creates schedules, a heartbeat monitor, or a workflow scheduler such as Airflow. The manager does not remove the need to define idempotency, retries, logs, ownership, and alerting.

7.3 systemd timers and overlap protection

Systemd timers can express calendar schedules, persist missed runs, and connect naturally to service logs. For a job that must not overlap, use a lock:

```
flock -n /tmp/journey.lock \  
  /usr/bin/curl --fail --silent --show-error \  
  --max-time 10 https://example.com/health
```

`flock -n` exits immediately if another copy owns the lock. A production job should also record success and send a heartbeat to a monitor.

Create a cron job that appends the date to a disposable file, wait for two runs, then inspect it with `tail`. Move the same task to a systemd timer and compare the logs.

You can explain cron's five fields, the difference between cron and systemd timers, and why a scheduled job needs logging and locking.

Docker: package the application

Docker gives a process a repeatable filesystem, dependencies, environment, and network boundary. It does not replace Linux knowledge; it makes that knowledge easier to reproduce.

8.1 Image, container, volume, network

An image is a read-only template. A container is a running process created from that template. A volume stores data outside the container's writable layer. A Docker network gives containers a private way to find each other by service name.

```
docker version
docker run --rm hello-world
docker ps -a
```

`docker version` checks client and daemon. `docker run` creates and starts a container; `-rm` removes it after exit. `docker ps -a` lists running and stopped containers.

8.2 A Dockerfile, line by line

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["python", "app.py"]
```

`FROM` chooses the base filesystem. `WORKDIR` sets the process directory. `COPY` adds files. `RUN` executes during build, while `CMD` supplies the default runtime command. `EXPOSE` documents a port; it does not publish it.

```
docker build -t journey-api:dev .
docker run --rm -p 8000:8000 journey-api:dev
curl --fail http://127.0.0.1:8000/health
```

The final dot is the build context. `-t` gives the image a name and tag. `-p 8000:8000` maps host port 8000 to container port 8000. The application must listen on the container network interface, not only on 127.0.0.1 inside the container.

8.3 Compose and persistence

Compose describes related services in one file, including ports, environment, dependencies, and volumes. Use `docker compose up -build` to build and start, `docker compose logs -f` to follow logs, and `docker compose down` to stop. Named volumes survive container replacement; they are not backups.

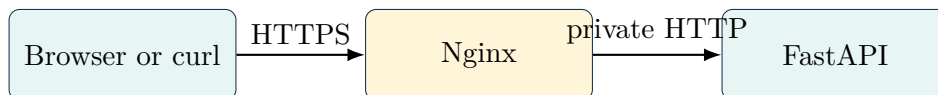
Check `docker ps`, then `docker logs`, then the published port, then the process's bind address, then the dependency network. Change one variable at a time.

You can distinguish an image, container, volume, and network, and you know what each Dockerfile instruction does.

Nginx, HTTPS, and the edge

Nginx is the public-facing door. It can serve static files, terminate TLS, choose an upstream application, and provide a stable place for logs and headers.

9.1 Reverse proxy mental model



The public client sees one stable hostname. Nginx handles certificates and forwards an internal request to the application. The application does not need to be directly reachable from the internet.

9.2 A minimal server block

```
server {  
    listen 80;  
    server_name example.com;  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

`listen` chooses the public port. `server_name` selects the hostname. `location /` matches every path. `proxy_pass` forwards the request. Forwarded headers preserve the original host and client chain.

```
sudo nginx -t  
sudo systemctl reload nginx  
curl -I http://example.com
```

Always test configuration before reloading. A reload is usually less disruptive than a restart. Add HTTPS only after DNS, port 80, and the application path work predictably.

9.3 TLS and secrets

TLS provides encryption and server identity. Certificate renewal is an operational process, not a one-time checkbox. Keep private keys outside Git, inject application secrets through a controlled mechanism, and check which users and processes can read them.

You can describe the request path through Nginx, test configuration before reload, and explain why TLS verification and secret storage matter.

Git, deployment, and observability

Deployment is a controlled change. Git tells you what changed, a health check tells you whether the service works, and logs and metrics tell you why it does not.

10.1 A small Git loop

```
git status
git diff
git add app.py
git commit -m "Add health endpoint"
git log --oneline -5
```

`git status` is the inventory. `git diff` is the evidence. `git add` selects exact files. The commit records a recoverable change. Read the diff before staging; Git does not understand whether a secret is safe to commit.

10.2 Health, readiness, and logs

A liveness check asks “is the process alive?” A readiness check asks “can it serve useful traffic?” Use curl with a timeout and failure handling; record response time and status.

```
curl --fail --silent --show-error --max-time 5 \
  --write-out '\nstatus=%{http_code} time=%{time_total}s\n' \
  http://127.0.0.1:8000/health
```

The `-write-out` template prints metadata after the body. A status of 200 is useful evidence, while timing helps reveal a slow dependency. Keep secrets out of command lines because shell history and process listings can expose them.

10.3 Backups and rollback

A backup is only real after a restore test. A rollback is only real after you know which image, commit, or package set was previously healthy. Write the procedure before the incident and practise it on the VM.

Capstone: a small service from laptop to production shape

This final project joins the layers. You will build a tiny API, test it with curl, package it with Docker, schedule a maintenance check, and expose it through Nginx.

11.1 Build the application

Create a disposable folder and a minimal FastAPI service. Run it locally first, then request `/health` with curl. Record the process ID, listening address, and response status. The application is intentionally small so the operational steps remain visible.

11.2 Containerise and compose

Build the image, run it on a private port, and check logs. Add a Compose file with an application service and a named volume only if the application needs persistent data. Use service names for internal discovery, not hard-coded container IP addresses.

11.3 Put Nginx at the edge

Configure a server block, test it, reload it, and verify the complete path: client to Nginx, Nginx to the container, application response, and logs at both layers. Only then add DNS and TLS.

11.4 Schedule an operational task

Create either a cron entry or systemd timer that calls the health endpoint with curl, uses a timeout, writes a timestamped log, and sends a heartbeat. Add a lock so two copies cannot overlap. Stop the application deliberately and observe the failure evidence.

11.5 Fault-injection checklist

1. Use the wrong hostname and identify the DNS symptom.
2. Stop the container and identify the connection symptom.
3. Break the Nginx upstream port and read both logs.

4. Fill a disposable log file and identify the disk symptom.
5. Run the scheduled check with a missing environment variable and fix its non-interactive environment.

You can explain the request path, process owner, network boundary, scheduler, logs, and rollback point for your service.

+

Worked examples: command to evidence

The fastest way to become comfortable is to read a command and its result as one story. These scenarios deliberately show the reasoning between typing and deciding.

12.1 Example: a health check that fails correctly

Suppose an application exposes `/health`. Start with a readable interactive check:

```
curl --fail --silent --show-error \  
  --max-time 5 \  
  --write-out '\nstatus=%{http_code} time=%{time_total}s\n' \  
  http://127.0.0.1:8000/health  
echo $?
```

A healthy response might look like:

```
{"status":"ok"}  
status=200 time=0.018421s  
0
```

The JSON body is application data. The status line is curl metadata. The final zero is the shell exit status. If the server returns 503, `-fail` makes curl return non-zero even if a body arrives. If the TCP connection is refused, there is no HTTP status at all. That difference tells you whether to inspect the application or the listening socket.

Stop the local service and run the command again. Write down the exact error. Start the service and make the same request. Compare the two results without changing the curl command.

12.2 Example: turn an API response into a small report

```
curl --fail --silent https://api.github.com/repos/curl/curl |  
jq '{name, stars: .stargazers_count, issues: .open_issues_count}'
```

The pipe is a boundary. `curl` owns transport and HTTP failure handling; `jq` owns JSON selection. If the API returns an error object, `jq` may still produce output unless `curl` is configured to fail. If a field is missing, `jq` can return null, which is evidence that the response shape changed.

```
{
  "name": "curl",
  "stars": 35000,
  "issues": 120
}
```

The numbers above are illustrative. The lesson is to name fields explicitly and record when the request was made.

12.3 Example: diagnose a port in layers

```
getent hosts example.com
nc -vz example.com 443
curl -vI https://example.com
```

If name lookup fails, the failure is before a TCP connection. If lookup succeeds but `nc` reports connection refused, the address is reachable but no process accepts that port. If TCP succeeds and `curl` fails during TLS, inspect the certificate name, clock, trust store, or proxy. If TLS succeeds and the response is 404, the network is working; the path or application contract is wrong.

12.4 Example: make a scheduled job observable

A weak cron entry hides its environment and writes nothing:

```
*/5 * * * * backup.sh
```

A stronger practice entry is explicit:

```
*/5 * * * * /usr/bin/flock -n /tmp/backup.lock \
  /usr/bin/curl --fail --silent --show-error --max-time 20 \
  --output /dev/null https://example.com/health \
  >> /home/ubuntu/journey/notes/health.log 2>&1
```

The absolute paths avoid cron's reduced PATH. `flock` prevents overlap. `-output /dev/null` discards the body while preserving the exit status. Redirection records both streams. A real operation adds a heartbeat monitor and a retention policy for the log.

12.5 Example: inspect a failed container

```
docker ps
docker ps -a
docker logs --tail 80 journey-api
```

```
docker inspect journey-api --format '{{.State.Status}} {{.State.ExitCode}}'
```

Start with inventory. If the container is absent, inspect the Compose project or image name. If it is exited, logs and exit code explain the startup failure. If it is running but curl cannot connect, inspect published ports and the application's bind address. Do not delete the container until you have captured useful logs.

12.6 Example: validate an Nginx change

```
sudo nginx -t
sudo systemctl status nginx --no-pager
sudo systemctl reload nginx
curl -I https://example.com
```

The order matters. Syntax validation protects the current service. Status shows whether the manager believes the service is healthy. Reload applies the new configuration with less disruption than a restart. curl proves the public path, not merely that the configuration parser succeeded.

For each scenario, you can name the layer that produced the evidence and the next command that would reduce uncertainty.

Practice labs

Use these labs after the main chapters. Each lab has a starting state, a small goal, and a definition of done. Keep all files under `~/journey/lab`.

13.1 Lab 1: build a command diary

Create `notes/commands.md`. For ten commands, record the problem, the command, the important option, and the exit status. Include one command that intentionally fails. This turns “I ran it” into a reusable explanation.

13.2 Lab 2: compare editors

Create the same three-line file in Nano and Vim. In Nano, search for a word and save. In Vim, insert a line, undo it, search for a word, save, and quit. Write down the mode or key sequence that felt least obvious.

13.3 Lab 3: a media inspection report

Use `ffprobe` on a video you are allowed to use. Record duration, dimensions, video codec, audio codec, and frame rate. Extract audio without re-encoding, then create a smaller preview with a filter. Compare file sizes and explain the quality trade-off.

13.4 Lab 4: find a slow process

Start a long sleep in the background. Find it with `ps`, inspect its process ID, send `SIGTERM`, and confirm it disappeared. Repeat with a harmless script that writes a timestamp, then observe its output with `tail -f`.

13.5 Lab 5: SSH between two terminals

Use the VM’s own address or a second disposable VM. Generate a key, install the public key, connect with a host alias, and copy a file with `rsync` dry-run first. Deliberately use the wrong port and explain the timeout.

13.6 Lab 6: cron versus systemd timer

Schedule a timestamp every two minutes with cron. Record where the output goes. Recreate the same task as a systemd service and timer. Compare status output, logs, missed-run behavior, and how each task is disabled.

13.7 Lab 7: container boundary

Build a tiny HTTP application. Run it with a published port, then run it without a published port. From the host, explain why curl succeeds in one case and fails in the other. Add a named volume and prove which data survives container replacement.

13.8 Lab 8: reverse proxy evidence

Place Nginx in front of the container. Request the public path and inspect Nginx and application logs. Break the upstream port, make one request, and identify the corresponding entries at both layers. Restore the port and repeat the health check.

13.9 Lab 9: restore rehearsal

Copy a small database or JSON file into a backup directory, record a checksum, delete the working copy, and restore it. Verify the checksum. The exercise is complete only when you can describe the restore command without looking it up.

13.10 Lab 10: clean-snapshot capstone

Return to the clean VM snapshot and repeat the capstone using only your notes. When you get stuck, write the question before searching. Update the notes with the evidence that solved it.

A lab is complete when the command worked, the output was interpreted, the system was returned to a known state, and the procedure is understandable to another beginner.

Command cards

Command	First question it helps answer
<code>pwd, ls -la</code>	Where am I and what is here?
<code>rg, find</code>	Which text or files match?
<code>ps, top</code>	Which process is consuming time or memory?
<code>df -h, free -h</code>	Is the machine running out of capacity?
<code>ip, ss, dig</code>	Where does the network path fail?
<code>curl -v</code>	What happened during DNS, TLS, and HTTP?
<code>ssh, scp, rsync</code>	Can I reach and transfer to the remote machine?
<code>systemctl, journalctl</code>	Is the service running and why?
<code>docker ps, docker logs</code>	Is the container alive and what did it print?
<code>nginx -t</code>	Is the proxy configuration syntactically valid?

Troubleshooting map

When a URL fails, write down the exact command, timestamp, hostname, port, and error. Then test one layer at a time: local process, listening socket, route, DNS, TCP connection, TLS certificate, HTTP status, application dependency, and scheduled environment. Do not change three layers at once; you will lose the evidence that identifies the cause.

A thirty-day practice path

Practise for 20–30 minutes a day. Days 1–3 cover VM safety and terminal navigation. Days 4–6 cover curl and JSON. Days 7–10 cover files and editors. Days 11–14 cover permissions, processes, and logs. Days 15–18 cover networking and SSH. Days 19–21 cover cron and systemd. Days 22–25 cover Docker. Days 26–28 cover Nginx and TLS. Days 29–30 repeat the capstone from a clean snapshot without looking at the commands.

Glossary and official references

API	A defined interface through which programs exchange data.
Daemon	A background process that provides a service.
DNS	The naming system that maps hostnames to addresses.
Exit status	The numeric result a process returns to its caller.
Image	A read-only filesystem template used to create containers.
Reverse proxy	A server that accepts client requests and forwards them to another service.
Socket	An endpoint used by a process to communicate over a network.
Systemd	A Linux service manager and boot-time supervisor.
VM	A virtual machine that runs an operating system inside another system.

Read the current official documentation when syntax or behavior matters: <https://curl.se/docs/>, <https://ffmpeg.org/documentation.html>, <https://ubuntu.com/server/docs>, <https://www.openssh.com/manual.html>, <https://docs.docker.com/>, <https://docs.nginx.com/>, and <https://www.freedesktop.org/wiki/Software/systemd/>.

Closing note

The goal is not to memorise a thousand commands. The goal is to form a reliable loop: make a small change, observe the result, explain the evidence, and leave the system in a known state. That loop is the foundation beneath Linux administration, backend engineering, DevOps, and data engineering.